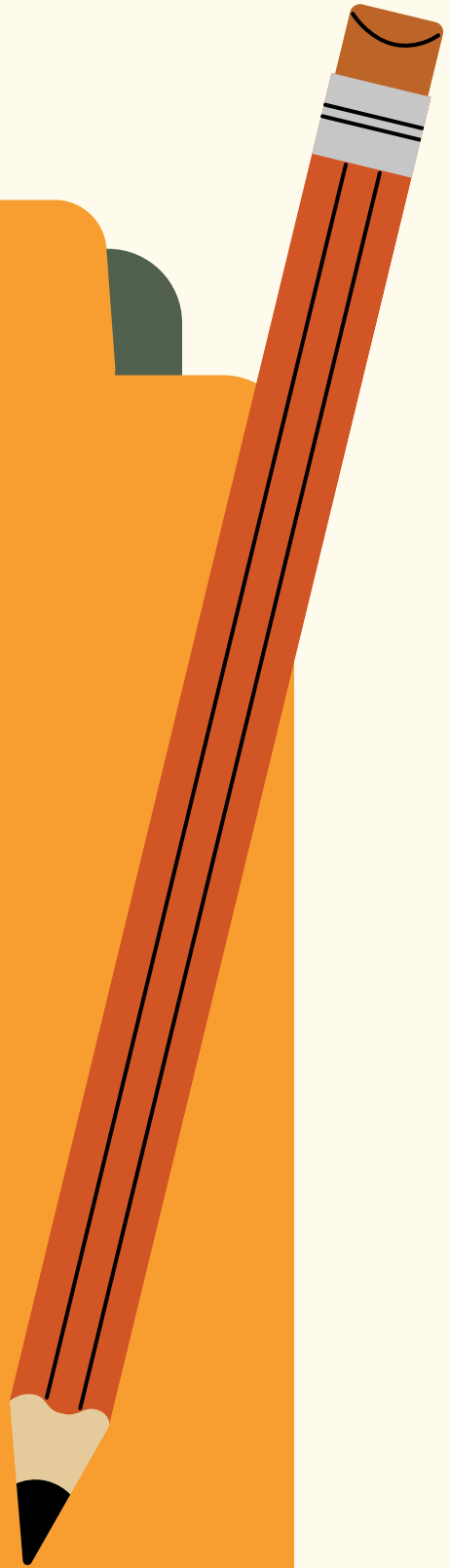


From numbers to AI

PROGRAMMING

파이썬





질문있나요?

<https://discord.gg/mQa86T9NU2>

cm@playidealab.com 또는 lcmin@yonsei.ac.kr

모르는건 당연한거지만, 모르는채로 있는건 선택이지요

질문하는 과정에서 깨닫기도 합니다



Library

Matplotlib, Numpy, Pandas



Library

Library > Package > module

학습목표

- Library 알아보기, 설치하기
- 다양한 라이브러리 종류 알아보기
- Library 응용

Library?

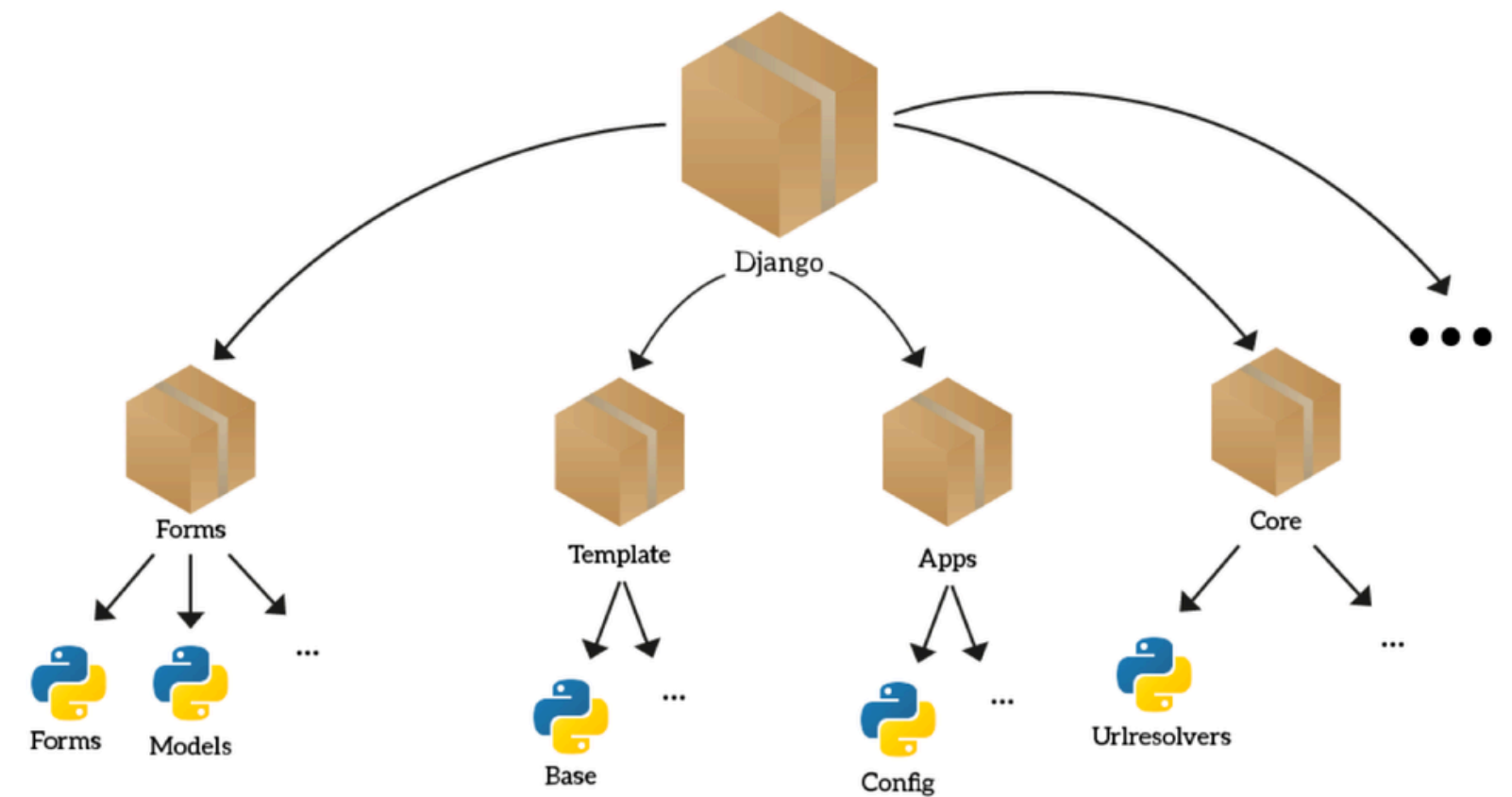
- 도서관의 책 : 다양한 정보
- **필요한** 정보를 **원할 때** 찾아볼 수 있다
- **필요한** 기능을 **원할 때** 찾아 쓸 수 있는 건 없나?



Library, Module, Package

- Function, class : 기능을 구현
- Module : method(function) + variable + class
일반적으로 파일 1개로 구성
ex.) file.py.
- Package : 특정 기능과 관련된 여러 모듈이 모인 폴더
- Library : 패키지 모음

```
import [module]  
from [package] import [module]
```



Standard Library

Python 과 함께 배포되는 표준 라이브러리

Standard Library

- 파이썬과 함께 배포되는 표준 라이브러리
- Python 공식 문서 설명 : <https://docs.python.org/ko/3/library/index.html>

- 소거 - 가용성에 대한 참고 사항 - 내장 함수 - 내장 상수 - site 모듈에 의해 추가된 상수들 - 내장명 - 논리곱 연산 - 논리 연산 — and, or, not - 비교 - 숫자형 — int, float, complex - 이터러블의 길 - 시퀀스 형 — list, tuple, range - 텍스트 시퀀스 형 — str - 비이터블 시퀀스 형 — bytes, bytearray, memoryview - 집합 형 — set, frozenset - 매핑 형 — dict - 선택세트 관리자 형 - Type Annotation Types — Generic Alias, Union - 기타 내장형 - 특수 어트리뷰트 - 내장 예외 - Exception classed - Inheriting from built-in exceptions - 베이스 클래스 - 구체적인 예외 - 경고 - 예외 계층 구조 - 텍스트 처리 서비스 - string — 일반적인 문자열 연산 - re — 정규식 연산 - diff — 파일 계산을 위한 도구임 - textwrap — 텍스트 래핑과 채우기 - unicodedata — 유니코드 데이터베이스 - stringprep — 인터넷 문자열 준비 - readline — GNU readline 인터페이스 - rlcompleter — GNU readline을 위한 완성 함수 - 비이터블 데이터 서비스 - struct — 배열 및 비이터블 데이터로 바이트열을 처리 - cobblets — 코덱 레지스트리과 베이스 클래스 - 데이터형 - datetime — 기본 날짜와 시간 - zoneinfo — IANA 시대에 지원 - calendar — 달력 일력 관련 함수 - collections — 컨테이너 데이터형 - collections.abc — 컨테이너의 추상 베이스 클래스 - heapq — 힙 과 알고리즘 - bisect — 배열 이진 분할 알고리즘 - array — 효율적인 숫자 배열 - weakref — 약한 참조 - types — 동적 형 생성과 내장형 이름 - copy — 깊은 복사와 얕은 복사 연산 - operator — 연산 데이터 연산자 - reprlib — 다양한 repr() 구현 - enum — 열거형 지원 - graphlib — 그래프를 명시한 구조에 작동하는 기능 - 숫자와 수학 모듈 - numbers — 숫자 추상 베이스 클래스 - math — 수학 함수 - cmath — 복소수를 위한 수학 함수 - decimal — 십진 고정 소수점 및 부동 소수점 산술 - fractions — 유리수 - random — 의사 난수 생성 - statistics — 수학 통계 함수 - 함수형 프로그래밍 모듈 - itertools — 효율적인 무한을 위한 이터레이터를 만드는 함수 - functools — 고차 함수와 클로저를 처리에 대한 연산 - operator — 함수로서의 표준 연산자 - 파일과 디렉터리 액세스 - pathlib — 객체 지향 파일 시스템 경로 - os.path — 일반적인 경로용 조작	- fileinput — 여러 입력 스트림에서 줄을 읽어들이는 객체 - stat — stat() 결과 해석하기 - filecmp — 파일과 디렉터리 비교 - tempfile — 임시 파일과 디렉터리 생성 - glob — 유닉스 스타일 경로명 패턴 확장 - fnmatch — 유닉스 파일명 패턴 일치 - linecache — 텍스트 줄에 대한 무한된 액세스 - shutil — 고수준 파일 연산 - 데이터 지속성 - pickle — 파이썬 객체 직렬화 - copyreg — pickle 지원 함수 등록 - shelve — 파이썬 객체 지속성 - marshal — 내부 파이썬 객체 직렬화 - dbm — 유닉스 "데이터베이스" 인터페이스 - sqlite3 — SQLite 데이터베이스용 DB-API 2.0 인터페이스 - 데이터 압축 및 보관 - zlib — gzip 과 호환되는 압축 - gzip — gzip 파일 지원 - bz2 — bz2 압축 지원 - lzma — LZMA 알고리즘을 사용할 압축 - zipfile — ZIP 데이터의 직접 - tarfile — tar 데이터의 파일 읽기와 쓰기 - 파일 형식 - csv — CSV 파일 읽기와 쓰기 - configparser — 구성 파일 구문 분석기 - netrc — netrc 파일 처리 - plistlib — 애플 .plist 파일 생성과 구문 분석 - 일종의 서비스 - hashlib — 보안 해시와 메시지 요약 - hmac — 해시지 인증을 위한 키 해싱 - secrets — 비밀 정보를 위한 안전한 난수 생성 - 일반 운영 체제 서비스 - os — 고위 운영 체제 인터페이스 - io — 스트림 작업을 위한 핵심 도구 - time — 시간 액세스와 연산 - argparse — 명령행 옵션, 인자와 부속 명함을 위한 과서 - getopt — 명령 줄 옵션을 스킴을 구문 분석기 - logging — 파이썬 로깅 시스템 - logging.config — 로깅 구성 - logging.handlers — 로깅 처리기 - getpass — 암호를 읽는 암호 입력 - curses — 문자 열 디스플레이를 위한 터미널 처리 - curses.textpad — curses 로그를 위한 텍스트 입력 편집 - curses.ascii — ASCII 문자를 유틸리티 - curses.panel — curses 중 패널 스택 확장 - platform — 외부 플랫폼에 의해 데이터에 대한 액세스 - errno — 표준 errno 시스템 기호 - ctypes — 파이썬을 외부 함수 라이브러리 - 동시 실행 - threading — 스레드 기반 병렬 처리 - multiprocessing — 프로세스 기반 병렬 처리 - multiprocessing.shared_memory — 프로세스 간 직접 액세스를 위한 공유 메모리를 제공한다 - concurrent.futures — 병렬 작업 실행하기 - subprocess — 서브 프로세스 관리 - sched — 이벤트 스케줄러 - queue — 동기화된 큐 클래스 - concurrent.futures — 선택세트 변수 _thread — 저수준 스레드 API - 네트워크와 프로세스 간 통신 - asyncio — 비동기 I/O - socket — 저수준 네트워크 인터페이스 - ssl — 소켓 액세스용 TLS/SSL 래퍼 - select — IO 완료 대기 - selectors — 고수준 IO 다중화 - signal — 신호가 이벤트에 대한 처리가 설정 - mmap — 메모리 맵 파일 지원 - 인터넷 데이터 처리 - email — 전자 메일과 MIME 처리 패키지 - json — JSON 인코딩과 디코딩	- mailbox — 다양한 형식의 사서함 조작하기 - mimetypes — 파일명들 MIME 유형에 매핑 - base64 — Base64, Base32, Base64, Base65 데이터 인코딩 - binhex — binhex4 파일 인코딩과 디코딩 - binascii — 바이너리와 ASCII 간의 변환 - quopri — MIME quoted-printable 데이터 인코딩과 디코딩 - 구조화된 텍스트 처리 도구 - html — 하이퍼텍스트 태그를 읽어 지원 - html.parser — 간단한 HTML과 XHTML 구문 분석기 - html.entities — HTML 실용 엔타리의 정의 - XML 처리 모듈 - xml.etree.ElementTree — ElementTree XML API - xml.dom — 문서 처리 모듈 API - xml.dom.minidom — 최소 DOM 구현 - xml.dom.pulldom — 부분 DOM 트리 구축 지원 - xml.sax — SAX2 구문 분석기 지원 - xml.sax.handler — SAX 처리기의 베이스 클래스 - xml.sax.saxutils — SAX 유틸리티 - xml.sax.xmlreader — XML 구문 분석기 인터페이스 - xml.parsers.expat — Expat를 사용한 빠른 XML 구문 분석 - 인터넷 프로토콜과 지원 - webbrowser — Convenient web-browser controller - urllib — URL 처리 모듈 - urllib.request — URL을 찾기 위한 확장 가능한 라이브러리 - urllib.response — urllib가 사용하는 응답 클래스 - urllib.parse — URL을 구성 요소로 구분 분리 - urllib.error — urllib.request의 의해 발생하는 예외 클래스 - urllib.robotparser — robots.txt 구문 분석기 - http — HTTP 모듈 - http.client — HTTP 프로토콜 클라이언트 - ftplib — FTP 프로토콜 클라이언트 - poplib — POP3 프로토콜 클라이언트 - imaplib — IMAP4 프로토콜 클라이언트 - smtplib — SMTP 프로토콜 클라이언트 - uuid — RFC 4122 에 따른 UUID 생성 - socketserver — 네트워크 서버를 위한 모범패턴 - http.server — HTTP 서버 - http.cookies — HTTP 상태 관리 - http.cookiejar — HTTP 클라이언트를 위한 쿠키 처리 - xmlrpc — XMLRPC 서버와 클라이언트 모듈 - xmlrpc.client — XML-RPC 클라이언트 액세스 - xmlrpc.server — 기본 XML-RPC 서버 - ipaddress — IPv4/IPv6 호지 라이브러리 - 멀티미디어 서비스 - wave — WAV 파일 읽고 쓰기 - colorsys — 색 체계 간의 변환 - 각종 모듈 - gettext — 다국어 패키지 서비스 - locale — 지역화 서비스 - 프로그래밍 프레임워크 - turtle — 미용 그래픽 - cml — 줄 지음 명령 인터프리터 지원 - shlex — 간단한 어휘 분석 - Tk를 사용한 그래픽 사용자 인터페이스 - tkinter — Tk/Tk 라이브러리 인터페이스 - tkinter.colorchooser — 색상 선택 대화 상자 - tkinter.filedialog — Tkinter 파일 대화 - Tkinter 대화 상자 - tkinter.messagebox — Tkinter 메시지 포맷트 - tkinter.scrolledtext — 스크롤 되는 Text widget - tkinter.dnd — 드래그 앤드 드롭 지원 - tkinter.ttk — Tk 테마 유틸리티 - tkinter.tix — Extension widgets for Tk - IDLE - 개발 도구 - typing — 정적 타입 지원 - pydoc — 설명서 생성과 온라인 도움말 시스템 - 파이썬 개발 모드 - 파이썬 개발 모드의 효과 - ResourceWarning 예 - 잘못된 파일 기술자 예외 예 - doctest — 대화형 파이썬 예제 테스트 - unittest — 단위 테스트 프레임워크 - unittest.mock — 모의 객체 라이브러리 - unittest.mock — 시뮬레이션 2to3 - 파이썬 2에서 파이썬 3으로 자동 코드 변환 - test — 파이썬용 용어 테스트 패키지 - test.support — 파이썬 테스트 스위트용 유틸리티 - test.support.socket_helper — 파이썬 실행 테스트용 유틸리티 - test.support.bytecode_helper — 불어썬 바이트 코드 생성 테스트를 위한 지원 도구 - test.support.threading_helper — urllibs for threading tests - test.support.os_helper — urllibs for os tests - test.support.import_helper — urllibs for import tests - test.support.warnings_helper — urllibs for warnings tests - 디버깅과 프로파일링 - inspect — 모듈 검사 - bdb — 디버깅 프레임워크 - faulthandler — 파이썬 디버깅용 예외 - gc — 파이썬 디버깅 - 파이썬 프로파일러 - timeit — 작은 코드 조각의 실행 시간 측정 - trace — 파이썬 문장 실행 추적 - tracemalloc — 메모리 할당 추적 - 소도파악에 관련된 것 - distutils — 패키지 모듈 형식과 설치 - ensurepip — pip 설치 프로그램 부트스트랩 - venv — 가상 환경 생성 - zipapp — 실행 가능한 파이썬 zip 데이터로 관리 - 파이썬 실행 서비스 - sys — 시스템 특정 데이터와 함수 - sysconfig — 파이썬의 구성 정보에 접근하기 - builtins — 내장 객체 __main__ — Top-level code environment - warnings — 경고 제어 - dataclasses — 데이터 클래스 - contextlib — with 문 컨텍스트를 위한 유틸리티 - abc — 추상 베이스 클래스 - atexit — 종료 처리기 - traceback — 스택 트레이스액 인쇄와 호출 __future__ — 현재 문 형식 - gc — 가비지 수거기 인터페이스 - inspect — 라이브러리에 검사 - site — 사이트별 구성 용 - 사용자 정의 파이썬 인터프리터 - code — 인터프리터 베이스 클래스 - codeop — 파이썬 코드 컴파일 - 모듈 정보 모듈 - zipimport — Zip 저장소에서 모듈 임포트 - pkgutil — 패키지 확장 유틸리티 - modulefinder — 스크립트에서 사용되는 모듈 찾기 - runpy — 파이썬 모듈 찾기용 실행 - importlib — import의 구현 - importlib.metadata — 메타데이터 - 파이썬 언어 서비스 - ast — 추상 구문 트리 - syntable — 컴파일러 실행 데이터베이스 - token — 파이썬 구문 분석기에서 사용되는 상수 - keyword — 파이썬 키워드 검사 - tokenizer — 파이썬 소스를 위한 토큰라이저 - tabnanny — 코딩한 들여쓰기 관리 - pycparser — 파이썬 코딩 변환기 지원 - py_compile — 파이썬 소스 파일 컴파일 - compileall — 파이썬 라이브러리의 라이브러리 컴파일러 - difflib — 파이썬 버전의 코드 에 디펜더 - pickletools — 리눅스 개발자를 위한 도구 - 이동성 모듈 서비스 - msvcrt — MS VC++ 런타임의 동등한 루틴 - winreg — 윈도우 레지스트리 액세스
---	--	--

Standard Library

1. 운영 체제 인터페이스 - os 모듈

os 모듈은 운영 체제와 상호 작용하기 위한 수십 가지 함수들을 제공합니다:

```
>>> import os
>>> os.getcwd()      # Return the current working directory
'C:\Python310'
>>> os.chdir('/server/accesslogs')  # Change current working directory
>>> os.system('mkdir today')  # Run the command mkdir in the system shell
0
```

`from os import *` 대신에 `import os` 스타일을 사용해야 합니다. 그래야 `os.open()` 이 내장 `open()` 을 가리는 것을 피할 수 있는데, 두 함수는 아주 다르게 동작합니다.

Standard Library

1. 운영 체제 인터페이스 - os 모듈

os 와 같은 큰 모듈과 작업할 때, 내장 `dir()` 과 `help()` 함수는 대화형 도우미로 쓸모가 있습니다.

```
>>> import os
>>> dir(os)
<returns a list of all module functions>
>>> help(os)
<returns an extensive manual page created from the module's docstrings>
```

일상적인 파일과 디렉터리 관리 작업을 위해, `shutil` 모듈은 사용하기 쉬운 더 고수준의 인터페이스를 제공합니다:

```
>>> import shutil
>>> shutil.copyfile('data.db', 'archive.db')
'archive.db'
>>> shutil.move('/build/executables', 'installdir')
'installdir'
```

Standard Library

2. 파일 와일드 카드- glob

glob 모듈은 디렉터리 와일드카드 검색으로 파일 목록을 만드는 함수를 제공합니다:

```
>>> import glob
>>> glob.glob('*.py')
['primes.py', 'random.py', 'quote.py']
```

```
>>>
```

Standard Library

3. 명령행 인자 – sys, argparse

일반적인 유틸리티 스크립트는 종종 명령행 인자를 처리해야 할 필요가 있습니다. 이 인자들은 `sys` 모듈의 `argv` 어트리뷰트에 리스트로 저장됩니다. 예를 들어, 명령행에서 `python demo.py one two three` 를 실행하면 다음과 같은 결과가 출력됩니다:

```
>>> import sys
>>> print(sys.argv)
['demo.py', 'one', 'two', 'three']
```

`argparse` 모듈은 명령 줄 인자를 처리하는 더 정교한 메커니즘을 제공합니다. 다음 스크립트는 하나 이상의 파일명과 선택적으로 표시할 줄 수를 추출합니다:

```
import argparse

parser = argparse.ArgumentParser(
    prog='top',
    description='Show top lines from each file')
parser.add_argument('filenames', nargs='+')
parser.add_argument('-l', '--lines', type=int, default=10)
args = parser.parse_args()
print(args)
```

`python top.py --lines=5 alpha.txt beta.txt`를 사용하여 명령 줄에서 실행할 때, 스크립트는 `args.lines`를 5로, `args.filenames`를 `['alpha.txt', 'beta.txt']`로 설정합니다.

Standard Library

4. 에러 출력 리디렉션과 프로그램 종료

`sys` 모듈은 `stdin`, `stdout`, `stderr` 어트리뷰트도 갖고 있습니다. 가장 마지막 것은 `stdout` 이 리디렉트 되었을 때도 볼 수 있는 경고와 에러 메시지들을 출력하는데 쓸모가 있습니다:

```
>>> sys.stderr.write('Warning, log file not found starting a new one\n')
Warning, log file not found starting a new one
```

>>>

스크립트를 종료하는 가장 직접적인 방법은 `sys.exit()` 를 쓰는 것입니다.

Standard Library

5. 문자열 패턴 매칭

`re` 모듈은 고급 문자열 처리를 위한 정규식 도구들을 제공합니다. 복잡한 매칭과 조작을 위해, 정규식은 간결하고 최적화된 솔루션을 제공합니다:

```
>>> import re
>>> re.findall(r'\\b[a-z]*', 'which foot or hand fell fastest')
['foot', 'fell', 'fastest']
>>> re.sub(r'((\\b[a-z]+) \\1)', r'\\1', 'cat in the the hat')
'cat in the hat'
```

단지 간단한 기능만 필요한 경우에는, 문자열 메서드들이 선호되는데 읽기 쉽고 디버깅이 쉽기 때문입니다:

```
>>> 'tea for too'.replace('too', 'two')
'tea for two'
```

Standard Library

6. 수학

`math` 모듈은 부동 소수점 연산을 위한 하부 C 라이브러리 함수들에 대한 액세스를 제공합니다.

```
>>> import math
>>> math.cos(math.pi / 4)
0.70710678118654757
>>> math.log(1024, 2)
10.0
```

`random` 모듈은 무작위 선택을 할 수 있는 도구들을 제공합니다:

```
>>> import random
>>> random.choice(['apple', 'pear', 'banana'])
'apple'
>>> random.sample(range(100), 10)  # sampling without replacement
[30, 83, 16, 4, 8, 81, 41, 50, 18, 33]
>>> random.random()  # random float
0.17970987693706186
>>> random.randrange(6)  # random integer chosen from range(6)
4
```

Standard Library

6. 수학

`statistics` 모듈은 수치 데이터의 기본적인 통계적 특성들을 (평균, 중간값, 분산, 등등) 계산합니다.

```
>>> import statistics
>>> data = [2.75, 1.75, 1.25, 0.25, 0.5, 1.25, 3.5]
>>> statistics.mean(data)
1.6071428571428572
>>> statistics.median(data)
1.25
>>> statistics.variance(data)
1.3720238095238095
```

SciPy 프로젝트 <<https://scipy.org>> 는 다른 수치 계산용 모듈들을 많이 갖고 있습니다.

Standard Library

7. 인터넷 액세스

인터넷을 액세스하고 인터넷 프로토콜들을 처리하는 많은 모듈이 있습니다. 가장 간단한 두 개는 URL에서 데이터를 읽어오는 `urllib.request` 와 메일을 보내는 `smtplib` 입니다:

```
>>> from urllib.request import urlopen
>>> with urlopen('http://worldtimeapi.org/api/timezone/etc/UTC.txt') as response:
...     for line in response:
...         line = line.decode()           # Convert bytes to a str
...         if line.startswith('datetime'):
...             print(line.rstrip())       # Remove trailing newline
...
datetime: 2022-01-01T01:36:47.689215+00:00

>>> import smtplib
>>> server = smtplib.SMTP('localhost')
>>> server.sendmail('soothsayer@example.org', 'jcaesar@example.org',
...     """To: jcaesar@example.org
...     From: soothsayer@example.org
...
...     Beware the Ides of March.
...     """)
>>> server.quit()
```

(두 번째 예는 localhost 에서 메일 서버가 실행되고 있어야 한다는 것에 주의하세요.)

Standard Library

8. 날짜와 시간

`datetime` 모듈은 날짜와 시간을 조작하는 클래스들을 제공하는데, 간단한 방법과 복잡한 방법 모두 제공합니다. 날짜와 시간 산술이 지원되지만, 구현의 초점은 출력 포매팅과 조작을 위해 효율적으로 멤버를 추출하는 데에 맞춰져 있습니다. 모듈은 시간대를 고려하는 객체들도 지원합니다.

```
>>> # dates are easily constructed and formatted
>>> from datetime import date
>>> now = date.today()
>>> now
datetime.date(2003, 12, 2)
>>> now.strftime("%m-%d-%y. %d %b %Y is a %A on the %d day of %B.")
'12-02-03. 02 Dec 2003 is a Tuesday on the 02 day of December.'

>>> # dates support calendar arithmetic
>>> birthday = date(1964, 7, 31)
>>> age = now - birthday
>>> age.days
14368
```

Standard Library

9. 데이터 압축

일반적인 데이터 보관 및 압축 형식들을 다음과 같은 모듈들이 직접 지원합니다: `zlib`, `gzip`, `bz2`, `lzma`, `zipfile`, `tarfile`.

```
>>> import zlib
>>> s = b'witch which has which witches wrist watch'
>>> len(s)
41
>>> t = zlib.compress(s)
>>> len(t)
37
>>> zlib.decompress(t)
b'witch which has which witches wrist watch'
>>> zlib.crc32(s)
226805979
```

Standard Library

10. 성능 측정

일부 파이썬 사용자들은 같은 문제에 대한 다른 접근법들의 상대적인 성능을 파악하는데 깊은 관심을 두고 있습니다. 파이썬은 이런 질문들에 즉시 답을 주는 측정 도구를 제공합니다.

예를 들어, 인자들을 맞교환하는 전통적인 방식 대신에, 튜플 패킹과 언 패킹을 사용하고자 하는 유혹을 느낄 수 있습니다. `timeit` 모듈은 적당한 성능 이점을 신속하게 보여줍니다:

```
>>> from timeit import Timer
>>> Timer('t=a; a=b; b=t', 'a=1; b=2').timeit()
0.57535828626024577
>>> Timer('a,b = b,a', 'a=1; b=2').timeit()
0.54962537085770791
```

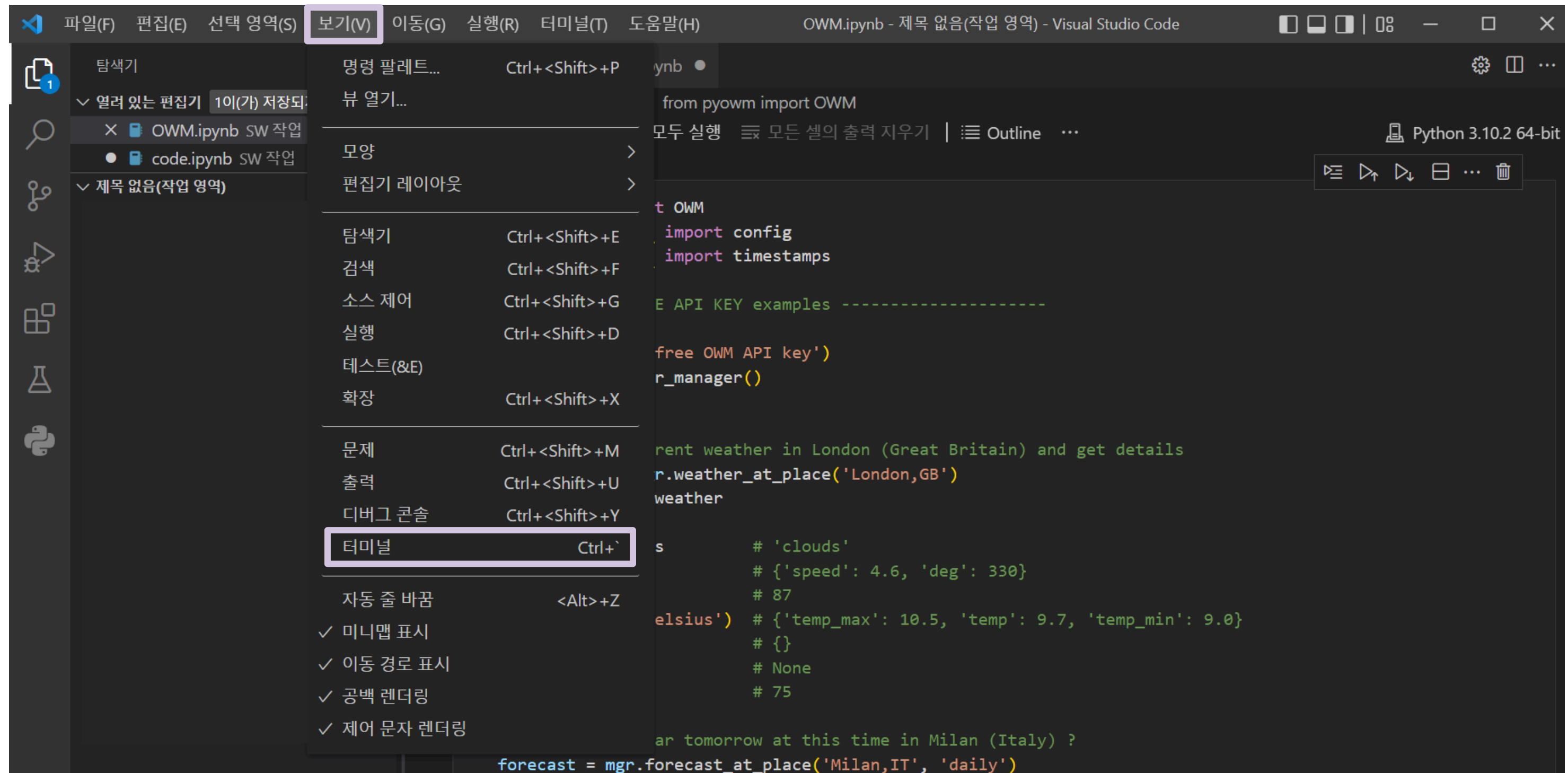
`timeit`의 정밀도와는 대조적으로, `profile` 과 `pstats` 모듈은 큰 블록의 코드에서 시간 임계 섹션을 식별하기 위한 도구들을 제공합니다.

유용한 Library

- 데이터 처리 : Pandas, Numpy, NetworkX
- 고속화 : Numba, Cython
- 딥러닝 : Pytorch, Tensorflow, Keras, Caffe, Theano, ScikitLearn, Scipy, Xgboost
- 이미지 및 영상 : Pillow, OpenCV, MoviePy
- 그래프 : Matplotlib, Plotly, Seaborn, Bokeh, ggplot2
- 크롤링 : BeautifulSoup, Selenium, Scrapy
- 백엔드, 웹 등 : Django, Requests, Bottle
- 게임 : PyGame
- Gui 등 : Tkinter, PyQt, Pyinstaller

How to install library

1. Visual Studio code



How to install library

1. Visual Studio code

```
# Will it be clear tomorrow at this time in Milan (Italy) ?
forecast = mgr.forecast_at_place('Milan,IT', 'daily')
answer = forecast.will_be_clear_at(timestamps.tomorrow())

# ----- PAID API KEY example -----

config_dict = config.get_default_config_for_subscription_type('professional')
owm = OWM('your paid OWM API key', config_dict)
```

문제 3 출력 디버그 콘솔 **터미널** JUPYTER powershell + - [] [X] ^ X

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 크로스 플랫폼 PowerShell 사용 <https://aka.ms/pscore6>

PS C:\Users\ \Desktop\SW 작업> **pip install pyowm**

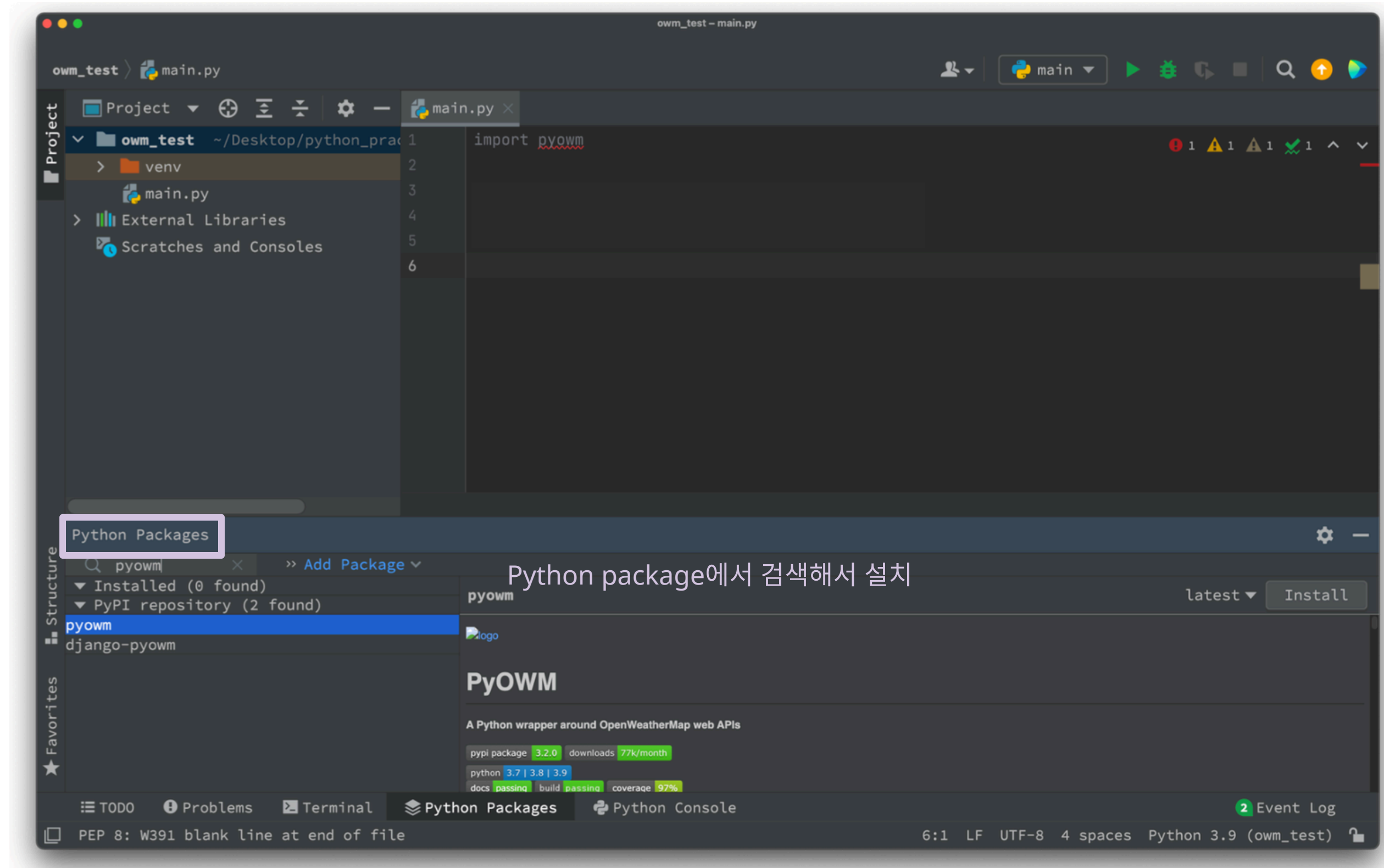
현재 작업 중인 위치가 맞는지 확인!

pip install "library 이름" (python version 2)
pip3 install "library 이름" (python version 3)

Jupyter Server: local

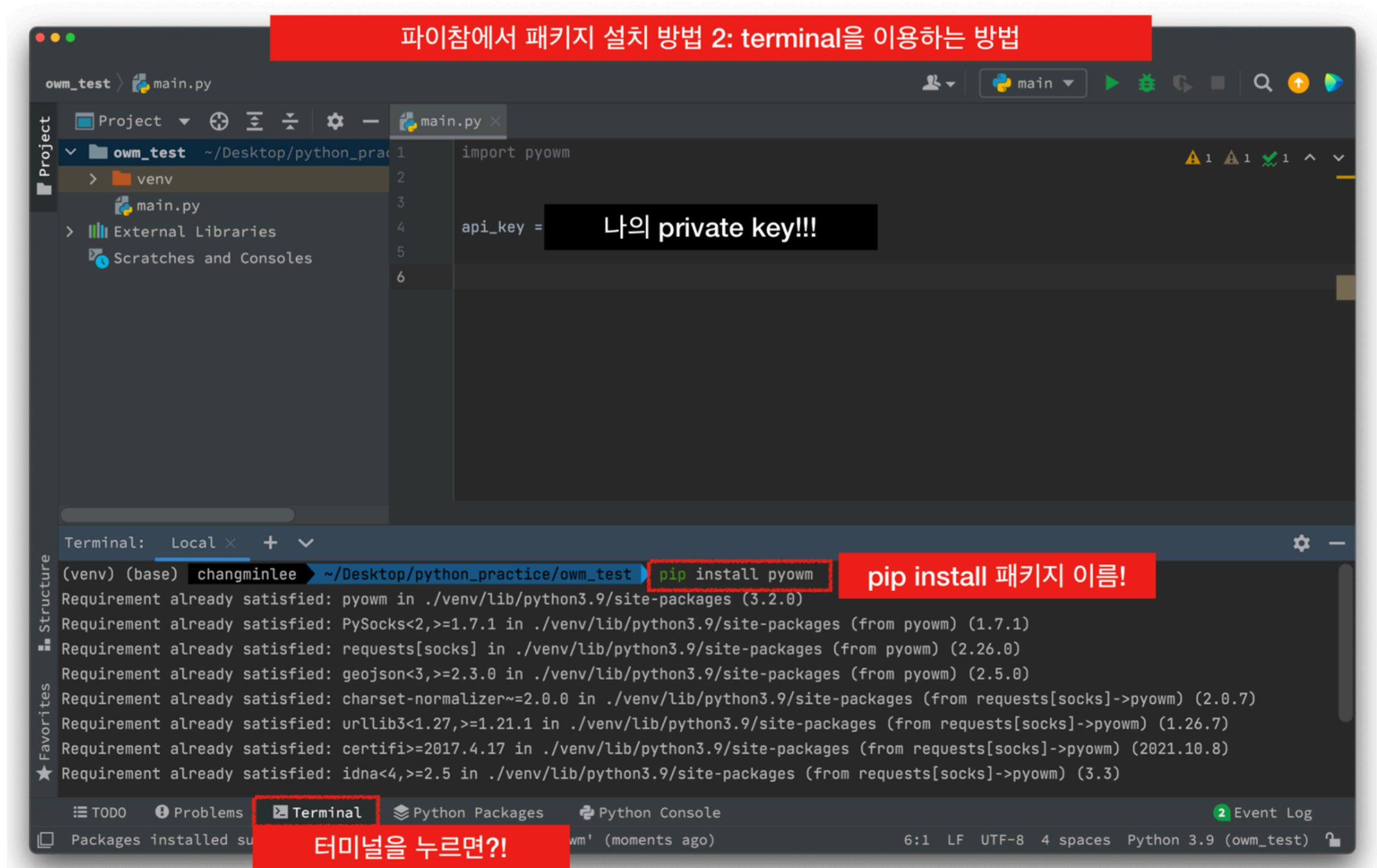
How to install library

2. Pycharm



How to install library

2. Pycharm



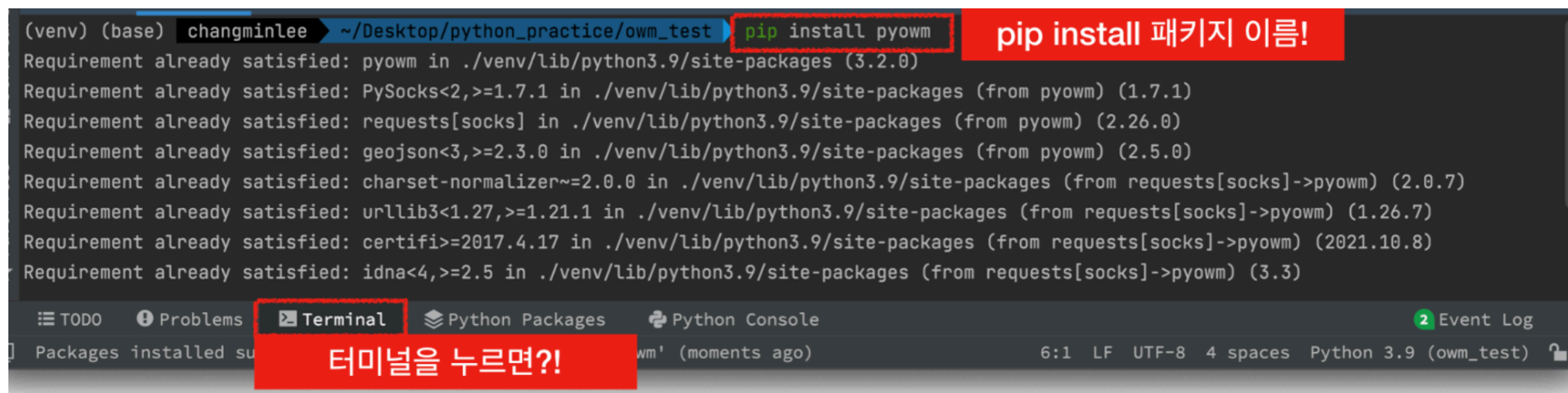
How to install library

3. colab

1 `!pip install numpy`

!로 시작하면 command line interface (CLI)임

애랑 같은거



```
(venv) (base) changminlee ~/Desktop/python_practice/owm_test pip install pyowm
Requirement already satisfied: pyowm in ./venv/lib/python3.9/site-packages (3.2.0)
Requirement already satisfied: PySocks<2,>=1.7.1 in ./venv/lib/python3.9/site-packages (from pyowm) (1.7.1)
Requirement already satisfied: requests[socks] in ./venv/lib/python3.9/site-packages (from pyowm) (2.26.0)
Requirement already satisfied: geojson<3,>=2.3.0 in ./venv/lib/python3.9/site-packages (from pyowm) (2.5.0)
Requirement already satisfied: charset-normalizer~=2.0.0 in ./venv/lib/python3.9/site-packages (from requests[socks]->pyowm) (2.0.7)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in ./venv/lib/python3.9/site-packages (from requests[socks]->pyowm) (1.26.7)
Requirement already satisfied: certifi>=2017.4.17 in ./venv/lib/python3.9/site-packages (from requests[socks]->pyowm) (2021.10.8)
Requirement already satisfied: idna<4,>=2.5 in ./venv/lib/python3.9/site-packages (from requests[socks]->pyowm) (3.3)
```

pip install 패키지 이름!

터미널을 누르면?!

Library

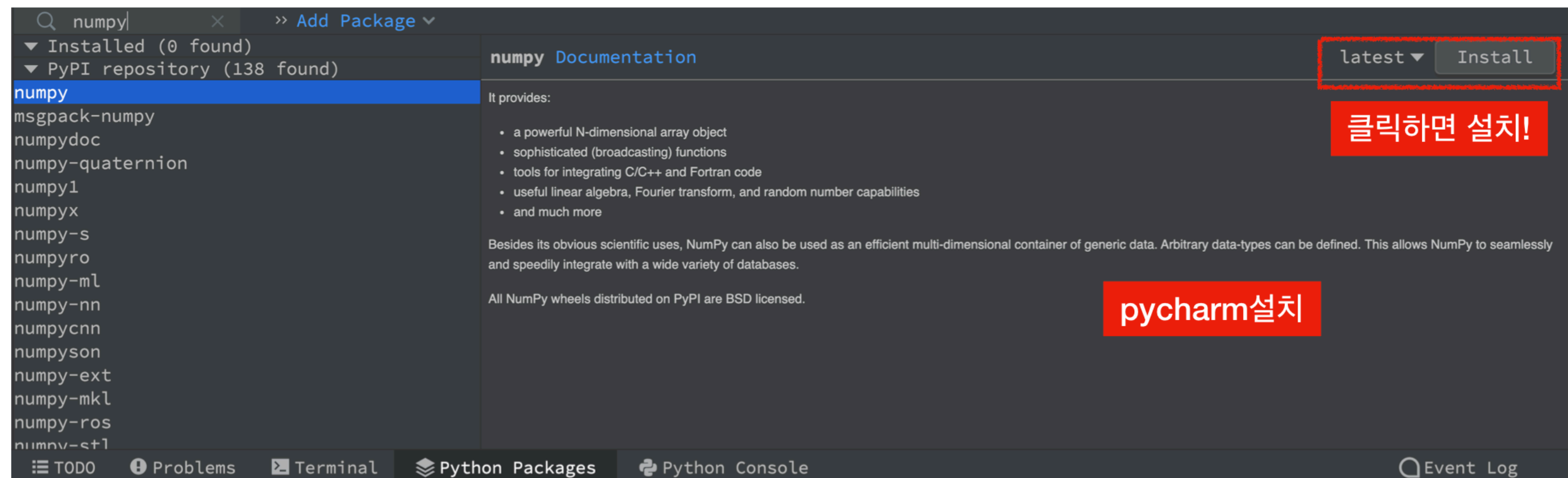
Numpy

Numpy

행렬을 다루는 기초

```
Terminal: Local x + v
(venv) (base) changminlee ~/Desktop/python_practice/owm_test pip install numpy
```

터미널에서 입력



```
[1]: !pip install numpy
Requirement already satisfied: numpy in /Users/changminlee/opt/anaconda3/lib/python3.8/site-packages (1.20.2)
```

Jupyter lab 등에서 설치



<https://numpy.org/>

How to use Numpy

공식 문서를 찾아보자



Python Numpy



python Numpy



로그인

전체

동영상

이미지

뉴스

쇼핑

더보기

도구

검색결과 약 92,300,000개 (0.37초)

<http://pythonstudy.xyz> > python > article > 402-numpy...

[numpy 사용하기 - 예제로 배우는 파이썬 프로그래밍](#)

예를 들어, 행이 2이고 열이 3인 2차원 배열에서 rank는 2 이고, shape는 (2, 3) 이 된다. numpy 배열을 생성하는 방법은 파이썬 리스트를 사용하는 방법과 numpy에서 제공 ...

<https://numpy.org>

NumPy

Nearly every scientist working in **Python** draws on the power of **NumPy**. **NumPy** brings the computational power of languages like C and Fortran to **Python**, a language ...

<http://bigdata.dongguk.ac.kr> > lectures > Python > _book

[4 장 Numpy | 파이썬 프로그래밍 기초](#)

4.2 배열의 생성. 리스트에서 행렬/배열 생성. `import numpy as np` # numpy 패키지 로드하여 `np` ...

<https://doorbw.tistory.com> > ...

[numpy\(넘파이\) 기초 정리](#)

2018. 6. 7. — **Numpy**는 C언어로 구현된 **파이썬** 라이브러리로써, 고성능의 수치계산을 위해 제작되었습니다. Numerical **Python**의 줄임말이기도 한 **Numpy**는 벡터 및 행렬 ...

<https://laboputer.github.io> > numpy-quickstart

[넘파이\(Numpy\) 사용법 알아보기 - 팽이쟁이](#)

2020. 4. 25. — 넘파이(**Numpy**)는 **Python**에서 벡터, 행렬 등 수치 연산을 수행하는 선형대수



넘파이

NumPy는 행렬이나 일반적으로 대규모 다차원 배열을 쉽게 처리 할 수 있도록 지원하는 파이썬의 라이브러리이다. NumPy는 데이터 구조 외에도 수치 계산을 위해 효율적으로 구현된 기능을 제공한다. [위키백과](#)

종류: 수치 해석

개발자: Community project

라이선스: 수정 BSD 라이선스

안정화 버전: 1.20.1 / 2021년 2월 7일

운영 체제: 크로스 플랫폼

프로그래밍 언어: 파이썬, C

관련 검색어 10개 이상 항목 더보기

How to use Numpy

공식 문서를 찾아보자

- Numpy > Document > API reference

Install

Documentation

Learn

Community

About Us

Contribute



The fundamental package for scientific computing with Python

GET STARTED

NumPy

NUMPY

NumFOCUS DEI research study - call for participation

POWERFUL N-DIMENSIONAL ARRAYS

Fast and versatile, the NumPy vectorization, indexing, and broadcasting concepts are the de-facto standards of array computing today.

NUMPY

NUMERICAL COMPUTING TOOLS

NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.

NUMPY

INTEROPERABLE

NumPy supports a wide range of hardware and computing platforms, and plays well with distributed, GPU, and sparse array libraries.

NUMPY

PERFORMANT

The core of NumPy is well-optimized C code. Enjoy the flexibility of Python with the speed of compiled code.

NUMPY

EASY TO USE

NumPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

NUMPY

OPEN SOURCE

Distributed under a liberal [BSD license](#), NumPy is developed and maintained [publicly on GitHub](#) by a vibrant, responsive, and diverse [community](#).

NUMPY

Try NumPy

Use the interactive shell to try NumPy in the browser

"""

To try the examples in the browser:

1. Type code in the input cell and press Shift + Enter to execute

Pyolite: A WebAssembly-powered Python kernel backed by Pyodide

[*]: `import numpy as np`



User Guide **API reference** Development Learn



Search the docs ...

Array objects

Constants

Universal functions (`ufunc`)

Routines

Typing (`numpy.typing`)

Global State

Packaging (`numpy.distutils`)

NumPy Distutils - Users Guide

NumPy C-API

SIMD Optimizations

NumPy and SWIG

NumPy Reference

Release: 1.22

Date: January 14, 2022

This reference manual details functions, modules, and objects included in NumPy, describing what they are and what they do. For learning how to use NumPy, see the [complete documentation](#).

- Array objects
 - The N-dimensional array (`ndarray`)
 - Scalars
 - Data type objects (`dtype`)
 - Indexing routines
 - Iterating Over Arrays
 - Standard array subclasses
 - Masked arrays
 - The Array Interface
 - Datetimes and Timedeltas
- Constants
- Universal functions (`ufunc`)
 - `ufunc`
 - Available ufuncs
- Routines
 - Array creation routines
 - Array manipulation routines
 - Binary operations
 - String operations
 - C-Types Foreign Function Interface (`numpy.ctypeslib`)
 - Datetime Support Functions
 - Data type routines

On this page

Acknowledgements

사용 방법을 알고 싶을 때

NumPy

User Guide API reference Development Learn

Search the docs ...

numpy.random.hypergeom

numpy.random.laplace

numpy.random.logistic

numpy.random.lognormal

numpy.random.logseries

numpy.random.multinomial

numpy.random.multivariate

numpy.random.negative_binomial

numpy.random.noncentral_chi

numpy.random.noncentral_f

numpy.random.normal

numpy.random.pareto

numpy.random.permutation

numpy.random.poisson

numpy.random.power

numpy.random.rand

numpy.random.randint

numpy.random.randn

numpy.random.random

numpy.random.random_integers

numpy.random.random_sample

numpy.random.ranf

numpy.random.rayleigh

numpy.random.sample

numpy.random.seed

numpy.random.set_state

numpy.random.shuffle

numpy.random.rand

random.rand(*d0*, *d1*, ..., *dn*)

Random values in a given shape.

Note

This is a convenience function for users porting code from Matlab, and wraps **random_sample**. That function takes a tuple to specify the size of the output, which is consistent with other NumPy functions like **numpy.zeros** and **numpy.ones**.

Create an array of the given shape and populate it with random samples from a uniform distribution over [0, 1).

Parameters: *d0*, *d1*, ..., *dn* : int, optional

The dimensions of the returned array, must be non-negative.
If no argument is given a single Python float is returned.

Returns: out : ndarray, shape (*d0*, *d1*, ..., *dn*)

Random values.

See also

random

Examples

>>> np.random.rand(3,2)
array([[0.14022471, 0.96360618], #random
 [0.37601032, 0.25528411], #random
 [0.49313049, 0.94909878]]) #random

< Previous

Next >

numpy.random.power

numpy.random.randint

- Numpy.random class 를 검색 : random의 method와 사용 방법이 나온다!

How to use Numpy

사용 방법을 알고 싶을 때

Google

numpy random

전체 동영상 이미지 뉴스 도서 더보기 도구

검색결과 약 18,300,000개 (0.24초)

<https://codetorial.net> > numpy > random ▾

NumPy 난수 생성 (Random 모듈) - Codetorial

`random.rand()` 함수는 주어진 형태의 난수 어레이를 생성합니다. 예제1 - 기본 사용법. `import numpy as np` a ...

<https://numpy.org> > reference > random > generated ▾

numpy.random.rand

`numpy.random.rand()` ... Random values in a given shape. ... This is a convenience function for users porting code from Matlab, and wraps `random_sample`. That ...

<https://aigong.tistory.com> > ... ▾

numpy random 모든 함수 살펴보기 (rand, randn, randint ...

2020. 5. 24. — **numpy.random** 대표 함수 살펴보기 (rand, randn, randint, random etc)

<https://docs.scipy.org/doc/numpy-1.15.0/reference/routines.random.html> 위 ...

NumPy

User Guide API reference Development Learn

numpy.random.rand

`random.rand(d0, d1, ..., dn)`

Random values in a given shape.

Note

This is a convenience function for users porting code from Matlab, and wraps `random_sample`. That function takes a tuple to specify the size of the output, which is consistent with other NumPy functions like `numpy.zeros` and `numpy.ones`.

Create an array of the given shape and populate it with random samples from a uniform distribution over `[0, 1)`.

Parameters: `d0, d1, ..., dn` : *int, optional*

The dimensions of the returned array, must be non-negative. If no argument is given a single Python float is returned.

Returns: out : *ndarray, shape (d0, d1, ..., dn)*

Random values.

See also

`random`

Examples

```
>>> np.random.rand(3,2)
array([[ 0.14022471,  0.96360618],  #random
       [ 0.37601032,  0.25528411],  #random
       [ 0.49313049,  0.94909878]]) #random
```

Previous `numpy.random.power` Next `numpy.random.randint`

- Numpy.random class 를 검색 : random의 method와 사용 방법이 나온다!

행렬

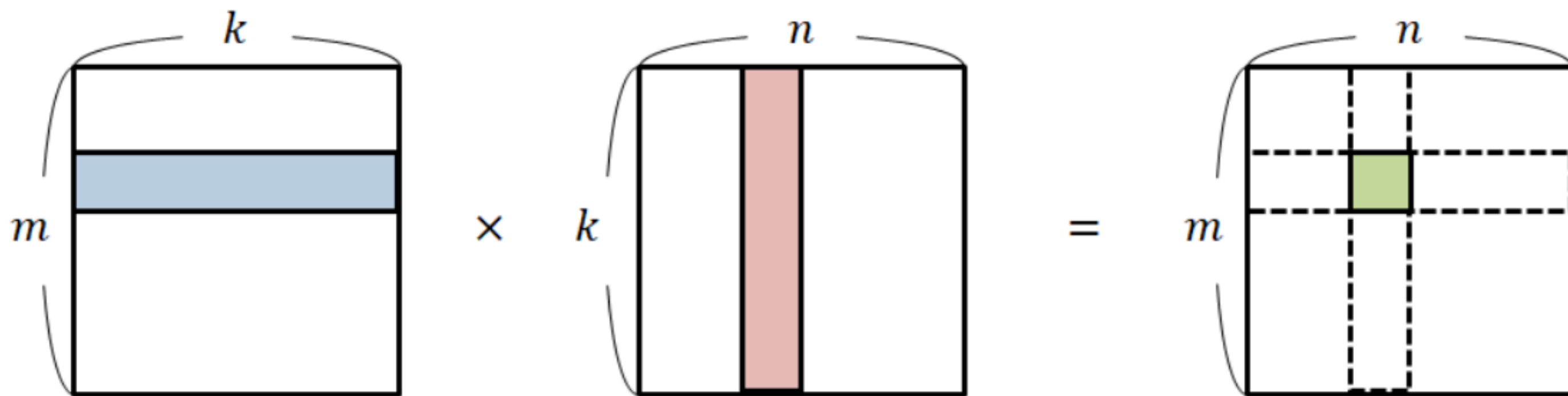


그림 2. 행렬의 일반적인 곱셈 방법

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1 \cdot a + 2 \cdot c & 1 \cdot b + 2 \cdot d \\ 3 \cdot a + 4 \cdot c & 3 \cdot b + 4 \cdot d \end{bmatrix}$$

Example of Numpy

행렬 생성, 편집

```
import numpy as np

list1 = [1, 2, 3, 4]
a = np.array(list1)    # list를 행렬로
print(a.shape)         # shape을 가지고 있음
print(list1.shape)     # list1.shape -> error
```

⊗ 0.6s

(4,)

```
-----
AttributeError                                Traceback (most recent call last)
Untitled-1.ipynb Cell 1' in <cell line: 6>()
      4 a = np.array(list1)    # list를 행렬로
      5 print(a.shape)         # list1.shape -> error
----> 6 print(list1.shape)

AttributeError: 'list' object has no attribute 'shape'
```

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)
print(b[0, 0])
```

✓ 0.5s

(2, 3)

1

- np.array(list) : list를 행렬로 변환

```
a = np.zeros((2, 2))    # 0으로 구성된 행렬
b = np.ones((2, 3))     # 1로 구성된 행렬
c = np.full((2, 3), 5)  # np.full((size), number) : number로 채워진 행렬
d = np.eye(3)           # 3x3 대각행렬
e = np.array(range(20)).reshape(4, 5)
# list를 행렬화 한 후, 4x5 행렬로 모양 변경
```

```
print(a, '\n\n', b, '\n\n', c, '\n\n', d, '\n\n', e)
```

✓ 0.4s

```
[[0. 0.]
 [0. 0.]]
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
[[5 5 5]
 [5 5 5]]
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
```

Example of Numpy

Indexing

```
lst = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
a = np.array(lst)
s = a[[0, 2], [1, 2]]
print(s)
```

✓ 0.5s

Python

[2 9]

```
bool_indexing_array = np.array([[False, True, False],\
                                [True, False, True], [False, True, False]])
```

```
n = a[bool_indexing_array]
print(n)
```

✓ 0.3s

Python

[2 4 6 8]

```
bool_indexing = (a % 2 == 0)    # 배열 a 에서 짝수는 True, 홀수는 False
print(a[bool_indexing])
print(a[a % 2 == 0])           # 간단한 표현
```

✓ 0.5s

Python

[2 4 6 8]

[2 4 6 8]

- 정수 인덱싱

: a[[0, 2], [1, 2]]

>>> [a[0, 1], a[2, 2]]

- Bool indexing

: Boolean array를 생성 후 True에 해당하는 값만 반환

Numpy function

array 1개의 각 성분에 적용되는 함수들

```
a = np.array([1, 4, 9, 16])
np.sqrt(a)
✓ 0.8s

array([1., 2., 3., 4.])

a = np.array([2.3, 3.6])
print(np.ceil(a))
print(np.floor(a))
✓ 0.4s

[3. 4.]
[2. 3.]
```

함수	설명
abs	각 성분의 절댓값 계산
sqrt	각 성분의 제곱근 계산 (array ** 0.5 의 결과와 동일)
square	각 성분의 제곱 계산 (array ** 2 의 결과와 동일)
exp	각 성분을 무리수 e의 지수로 삼은 값을 계산
log, log10, log2	자연로그(밑이 e), 상용로그(밑이 10), 밑이 2인 로그를 계산
sign	각 성분의 부호 계산 (+인 경우 1, -인 경우 -1, 0인 경우 0)
ceil	각 성분의 소수 첫 번째 자리에서 올림한 값을 계산
floor	각 성분의 소수 첫 번째 자리에서 내림한 값을 계산
isnan	각 성분이 NaN(Not a Number)인 경우 True를, 그렇지 않은 경우 False를 반환
isinf	각 성분이 무한대(infinity)인 경우 True를, 그렇지 않은 경우 False를 반환
cos, cosh, sin, sinh, tan, tanh	각 성분에 대한 삼각함수 값을 계산

Numpy function

array 2개의 각 성분 / 전체에 적용되는 함수들

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.add(a, b))
print(np.subtract(a, b))
print(np.multiply(a, b))
print(np.divide(a, b))
```

✓ 0.6s

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4 0.5 ]
```

함수	설명
add	두 array에서 동일한 위치의 성분끼리 더한 값을 계산 (arr1 + arr2의 결과와 동일)
subtract	두 array에서 동일한 위치의 성분끼리 뺀 값을 계산 (arr1 - arr2의 결과와 동일)
multiply	두 array에서 동일한 위치의 성분끼리 곱한 값을 계산 (arr1 * arr2의 결과와 동일)
divide	두 array에서 동일한 위치의 성분끼리 나눈 값을 계산 (arr1 / arr2의 결과와 동일)
maximum	두 array에서 동일한 위치의 성분끼리 비교하여 둘 중 최댓값을 반환
minimum	두 array에서 동일한 위치의 성분끼리 비교하여 둘 중 최솟값을 반환
np.dot	Np.multiply(a, b)의 모든 요소의 합
np.matmul	두 array 의 행렬 곱 (matrix product)

- `np.add(a, b) = a + b`
- `np.subtract(a, b) = a - b`

Numpy function

array에 적용되는 통계 함수

```
a = np.array([[1, 2],\
              [3, 4]])

print(np.sum(a))
print(np.sum(a, axis=1))
print(np.mean(a))
print(np.argmax(a))
```

✓ 0.9s

10
[3 7]
2.5
0

함수	설명
sum	전체 성분의 합을 계산
mean	전체 성분의 평균을 계산
std, var	전체 성분의 표준편차, 분산을 계산
min, max	전체 성분의 최솟값, 최댓값을 계산
argmin, argmax	전체 성분의 최솟값, 최댓값이 위치한 인덱스를 반환
cumsum	맨 첫번째 성분부터 각 성분까지의 누적합을 계산 (0에서부터 계속 더해짐)
cumprod	맨 첫번째 성분부터 각 성분까지의 누적곱을 계산 (1에서부터 계속 곱해짐)

- np.sum(array, axis)
 - : axis=1 : row sum
 - axis=0 : column sum
 - axis=None : sum of all element (default)

Library

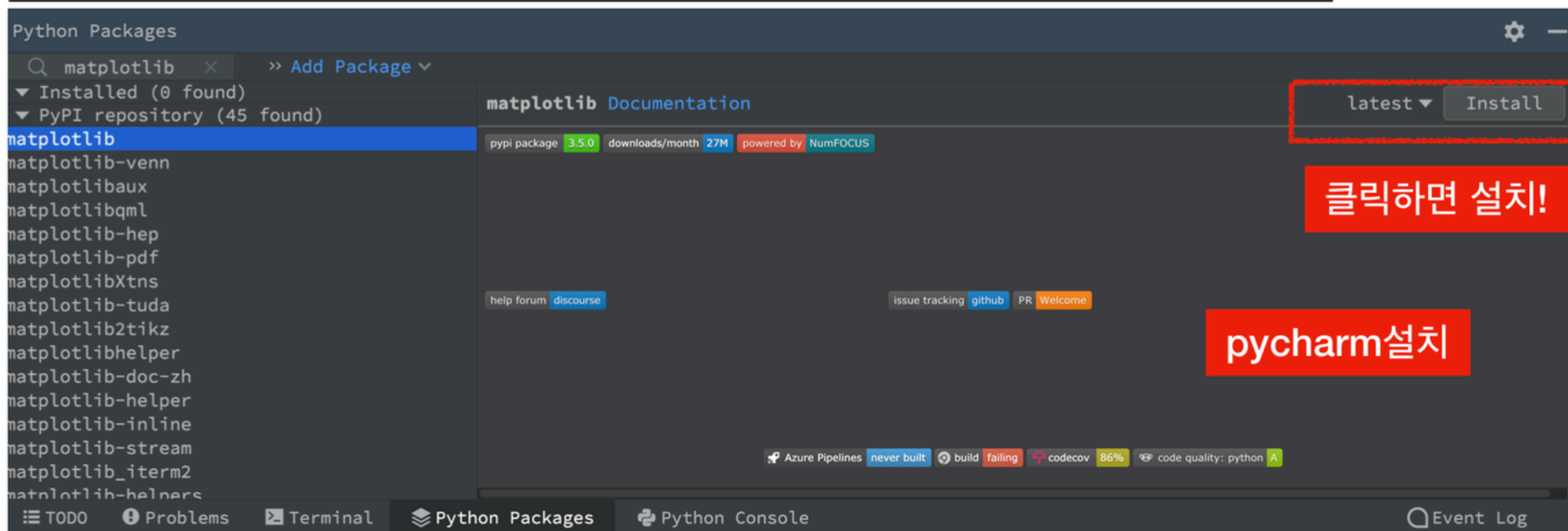
Matplotlib

Matplotlib

그래프 기본

```
Terminal: Local x + v
(venv) (base) changminlee ~/Desktop/python_practice/owm_test pip install matplotlib
Collecting matplotlib
  Downloading matplotlib-3.5.0-cp39-cp39-macosx_10_9_x86_64.whl (7.3 MB)
    | 7.3 MB 7.3 MB/s
```

터미널에서 입력



클릭하면 설치!

pycharm설치



<https://matplotlib.org/>

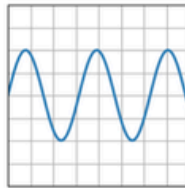
```
[2]: !pip install matplotlib
```

Jupyter lab 등에서 설치

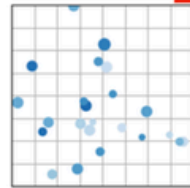
```
Requirement already satisfied: matplotlib in /Users/changminlee/opt/anaconda3/lib/python3.8/site-packages (3.3.4)
```

Matplotlib.pyplot 그래프 종류

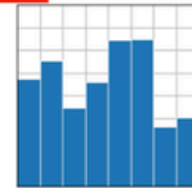
X vs Y



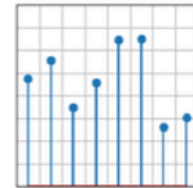
plot(x, y)



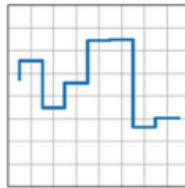
scatter(x, y)



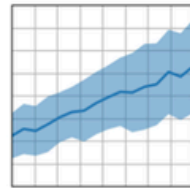
bar(x, height) / barh(y, width)



stem(x, y)

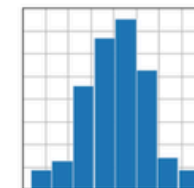


step(x, y)

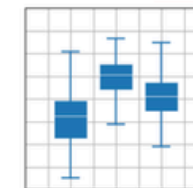


fill_between(x, y1, y2)

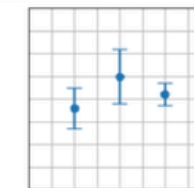
Statistics



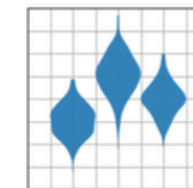
hist(x)



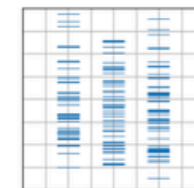
boxplot(X)



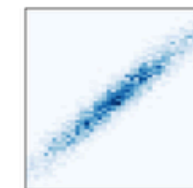
errorbar(x, y, yerr, xerr)



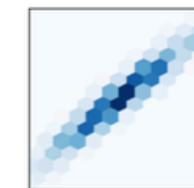
violinplot(D)



eventplot(D)



hist2d(x, y)

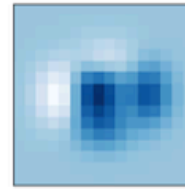


hexbin(x, y, C)

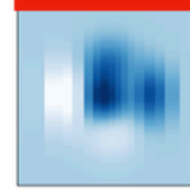


pie(x)

Plot arrays or fields



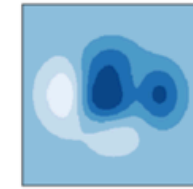
imshow(Z)



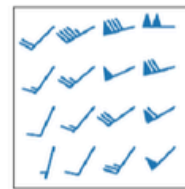
pcolormesh(X, Y, Z)



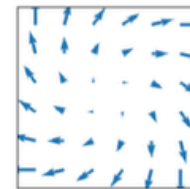
contour(X, Y, Z)



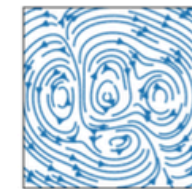
contourf(X, Y, Z)



barbs(X, Y, U, V)

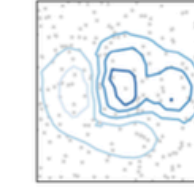


quiver(X, Y, U, V)

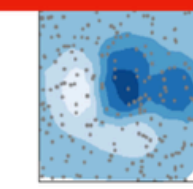


streamplot(X, Y, U, V)

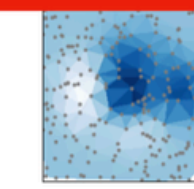
Unstructured coordinates



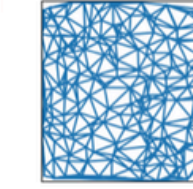
tricontour(x, y, z)



tricontourf(x, y, z)



tripcolor(x, y, z)



triplot(x, y)

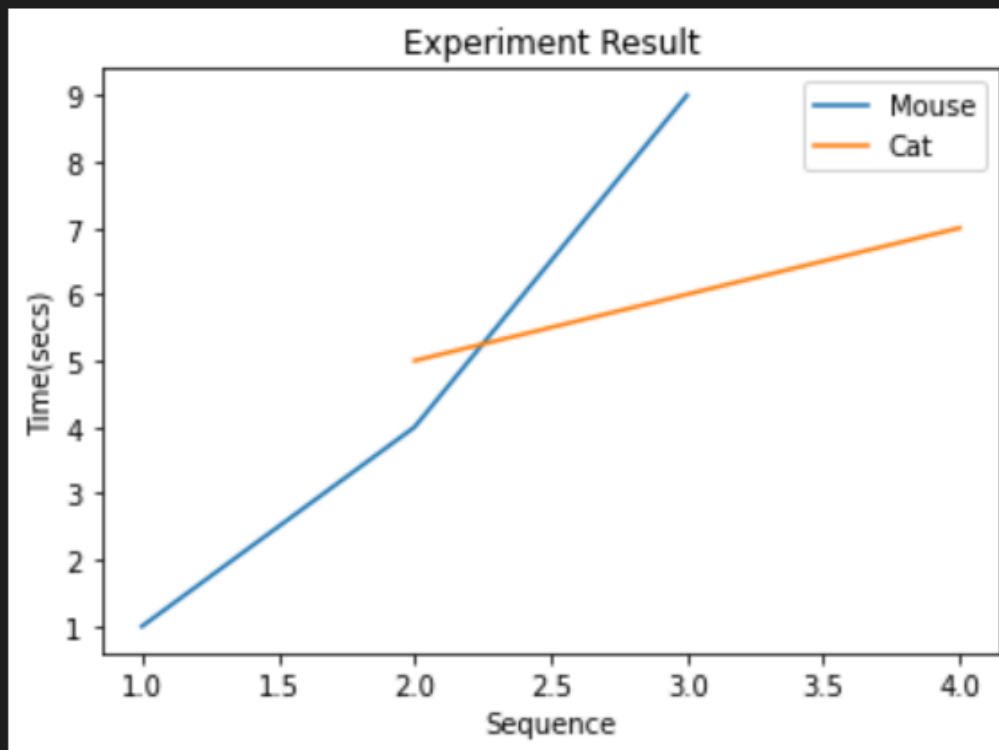
Example of Pyplotlib

Pyplot - 그래프 그리기

```
from matplotlib import pyplot as plt
```

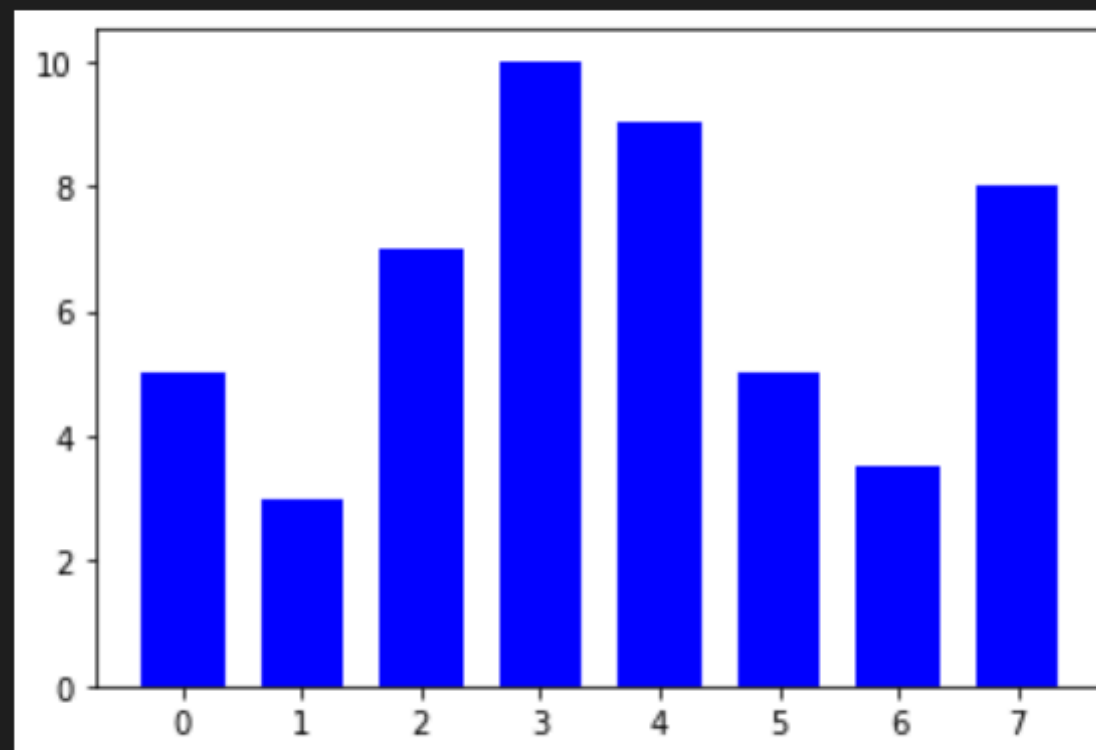
```
plt.plot([1, 2, 3], [1, 4, 9])  
plt.plot([2, 3, 4], [5, 6, 7])  
plt.xlabel('Sequence')  
plt.ylabel('Time(secs)')  
plt.title('Experiment Result')  
plt.legend(['Mouse', 'Cat'])  
plt.show()
```

✓ 1.2s



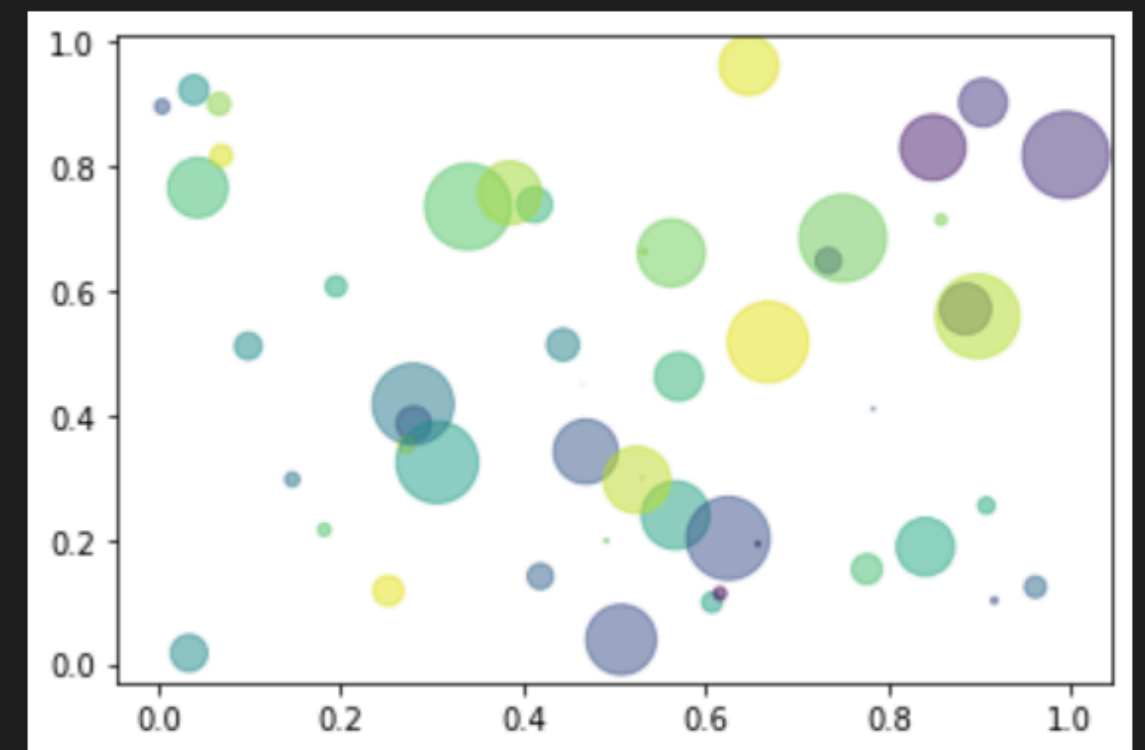
```
y = [5, 3, 7, 10, 9, 5, 3.5, 8]  
x = range(len(y))  
plt.bar(x, y, width=0.7, color='blue')  
plt.show()
```

✓ 0.1s



```
N = 50  
x = np.random.rand(N)  
y = np.random.rand(N)  
colors = np.random.rand(N)  
area = (30 * np.random.rand(N))**2  
  
plt.scatter(x, y, s=area, c=colors, alpha=0.5)  
plt.show()
```

✓ 0.1s



- 그래프에 제목, 범례, 축 이름, 그래프 색, 점/ 선 모양 등을 지정할 수 있다.

→ 데이터의 종류에 따라 그래프의 모양 선택

Example of Pyplotlib

<https://wonwooddo.tistory.com/68>

